Week 6

RAG & Agentic AI

AI Internship Training Program — Module 16 & 17

Learning Objectives

By the end of this week, interns will be able to:

- Understand how Retrieval-Augmented Generation (RAG) works.
- Build semantic search applications using embeddings.
- Store and retrieve embeddings using vector databases.
- Understand AI Agents and Agentic AI workflows.
- Build AI applications using FastAPI.
- Develop production-ready AI systems using modern frameworks.
- Create an intelligent Research Assistant capable of answering questions from documents.

Module 16: Retrieval-Augmented Generation (RAG)

Introduction

Large Language Models (LLMs) are powerful but have limitations:

- Knowledge cutoff
- Hallucinations
- Cannot access private documents
- Limited factual accuracy without external knowledge

Retrieval-Augmented Generation (RAG) solves these problems by retrieving relevant information from external documents before generating a response. Instead of relying only on the model's internal knowledge, RAG allows the model to answer questions using your own data.

RAG Pipeline

```
User Query
    |
    v
Embedding Model
    |
    v
Vector Database
    |
Retrieve Relevant Chunks
    |
    v
LLM
    |
    v
Final Response
```

Topics

1. Embeddings

Embeddings convert text into numerical vectors. Instead of storing words directly, AI stores their meaning in multidimensional space.

```
Dog  -> [0.21, -0.44, 0.83, ...]
```

Similar meanings produce similar vectors, for example King, Queen, Prince, and Princess will be located close together in vector space.

Generating an embedding in Python is one line of code once a model is loaded. Below, the sentence-transformers library turns two sentences into vectors, then cosine similarity tells us how close they are in meaning.

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

sentence1 = "The dog is playing in the park."
sentence2 = "A puppy is running outside."

emb1 = model.encode(sentence1, convert_to_tensor=True)
emb2 = model.encode(sentence2, convert_to_tensor=True)

similarity = util.cos_sim(emb1, emb2)
print("Similarity score:", similarity.item())
# Output close to 1.0 = very similar meaning
```

Why Embeddings?

- Semantic understanding
- Similarity search
- Recommendation systems
- Document retrieval
- Question answering

Popular Embedding Models

- sentence-transformers
- all-MiniLM-L6-v2
- BAAI BGE
- OpenAI Embeddings
- Gemini Embeddings
- E5 Embeddings

2. Chunking Strategies

LLMs cannot process extremely large documents efficiently. Therefore documents are split into chunks:

```
PDF  -> Page 1  -> Paragraphs  -> Chunks  -> Embeddings
```

Chunking Methods

Fixed Size Chunking

Splits text into equal-length pieces, e.g. 500 characters per chunk.

- Pros: Easy, Fast
- Cons: Breaks context

Recursive Chunking

Splits based on paragraph, sentence, and words — maintains better context.

Semantic Chunking

Uses AI to split by meaning. Most accurate approach.

Below is a practical example of Recursive Chunking using LangChain's text splitter. It tries to split on paragraphs first, then sentences, then words, so meaning stays intact as much as possible.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text = open("document.txt").read()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=600,       # target size per chunk (characters/tokens)
    chunk_overlap=100,    # overlap keeps context between chunks
    separators=["\n\n", "\n", ". ", " ", ""]
)

chunks = splitter.split_text(text)
print(f"Document split into {len(chunks)} chunks")
print(chunks[0])
```

Best Practices

- Chunk Size: 400–800 tokens
- Overlap: 50–150 tokens
- Preserve headings
- Avoid splitting tables

3. Vector Databases

A Vector Database stores embeddings instead of plain text. Instead of searching keywords, it searches meaning.

Traditional (SQL) Search:

```
SQL
WHERE name LIKE '%AI%'
```

Semantic Search:

```
"What is Artificial Intelligence?"

Finds:
  Machine Intelligence
  AI Systems
  Deep Learning
```

Popular Vector Databases

- FAISS
- ChromaDB
- Pinecone
- Weaviate
- Milvus
- Qdrant

A minimal FAISS example: build an index from a list of text chunks, then search it with a query.

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
chunks = ["AI is transforming healthcare.", "Cats are popular pets.", "Deep
learning powers modern AI."]

# Encode chunks into vectors and build an index
vectors = model.encode(chunks)
dimension = vectors.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(vectors))

# Search with a new query
query = "How is AI used in medicine?"
query_vector = model.encode([query])
distances, indices = index.search(np.array(query_vector), k=2)

for i in indices[0]:
    print(chunks[i])
```

4. Semantic Search

Semantic Search understands the meaning behind a query, matching by intent rather than exact words.

```
Query:
  How do neural networks learn?
Document:
  Backpropagation updates weights using gradients.
Keyword Search  -> No Match
Semantic Search -> Match Found
```

A full semantic search function that embeds the query, searches the vector database, and returns the closest matching chunks:

```
def semantic_search(query, index, chunks, model, top_k=3):
    query_vector = model.encode([query])
    distances, indices = index.search(query_vector, top_k)
```

```
    results = [chunks[i] for i in indices[0]]
    return results
top_matches = semantic_search("How do neural networks learn?", index, chunks,
model)
print(top_matches)
```

5. Hybrid Search

Hybrid Search combines Keyword Search (BM25) and Semantic Search (Embeddings), resulting in higher accuracy:

```
Final Score = 0.5 x Keyword Score + 0.5 x Semantic Score
```

Example combining BM25 keyword scores with embedding similarity scores:

```
from rank_bm25 import BM25Okapi

tokenized_chunks = [c.split() for c in chunks]
bm25 = BM25Okapi(tokenized_chunks)
def hybrid_search(query, alpha=0.5, top_k=3):
    # Keyword score
    bm25_scores = bm25.get_scores(query.split())

    # Semantic score (cosine similarity)
    query_vec = model.encode(query)
    semantic_scores = [
        util.cos_sim(query_vec, model.encode(c)).item() for c in chunks
    ]

    # Combine both scores
    final_scores = [
        alpha * bm25_scores[i] + (1 - alpha) * semantic_scores[i]
        for i in range(len(chunks))
    ]
    ranked = sorted(zip(chunks, final_scores), key=lambda x: x[1], reverse=True)
    return ranked[:top_k]
```

6. Reranking

Initial retrieval may return irrelevant documents. A Reranker sorts retrieved results based on relevance.

```
Retrieve Top 20
     |
     v
Reranker
     |
     v
Top 5
     |
     v
LLM
```

Popular Models:

- BGE Reranker
- Cross Encoder
- Cohere Rerank

Reranking the top 20 retrieved chunks down to the best 5 using a Cross Encoder:

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

pairs = [[query, chunk] for chunk in retrieved_top_20]
scores = reranker.predict(pairs)

reranked = sorted(zip(retrieved_top_20, scores), key=lambda x: x[1], reverse=True)
top_5 = [chunk for chunk, score in reranked[:5]]
```

Practical Tools

FAISS (Facebook AI Similarity Search)

Features:

- Extremely Fast
- Local Database
- Lightweight
- Ideal for prototypes

Operations:

- Create Index
- Store Embeddings
- Similarity Search

Saving and loading a FAISS index to disk so it persists between application runs:

```
# Save index
faiss.write_index(index, "my_index.faiss")

# Load index later
index = faiss.read_index("my_index.faiss")
```

ChromaDB

Open-source Vector Database

Features:

- Easy to use

- Metadata filtering
- Persistent storage
- LangChain Integration

Common Operations:

- Create Collection
- Add Documents
- Search
- Delete
- Update

A full ChromaDB example — create a persistent collection, add documents with metadata, and query it:

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(name="documents")

# Add documents (Chroma can auto-embed, or you can pass your own vectors)
collection.add(
    documents=["AI is transforming healthcare.", "Cats are popular pets."],
    metadatas=[{"source": "doc1.pdf"}, {"source": "doc2.pdf"}],
    ids=["id1", "id2"]
)

# Query the collection
results = collection.query(
    query_texts=["How is AI used in medicine?"],
    n_results=2
)
print(results["documents"])

# Update or delete
collection.update(ids=["id1"], documents=["Updated text about AI."])
collection.delete(ids=["id2"])
```

Module 17: Agentic AI Concepts & Applications

What is Agentic AI?

Traditional LLM:

```
Question  -> Answer
```

Agentic AI:

```
Question  -> Reason  -> Plan  -> Use Tools  -> Execute  -> Final Answer
```

AI Agents can think, plan, use tools, remember information, and complete tasks autonomously.

Topics

1. What is an AI Agent?

An AI Agent is a system that:

- Understands goals
- Plans tasks
- Makes decisions
- Uses external tools
- Learns from previous interactions
- Produces intelligent responses

Agent Workflow

```
Goal-> Planning -> Tool Selection -> Execution -> Observation  -> Next Action  -> Completion
```

2. Tool Calling

Agents become powerful by interacting with external tools, for example:

- Search API
- Calculator
- Weather API
- PDF Reader
- Database
- Email Sender
- Python Code Executor

Example Workflow:

```
User  -> Search Tool  -> Retrieved Data  -> LLM  ->Answer
```

A simple tool-calling agent using LangChain — the model decides on its own whether to use the calculator tool:

```
from langchain.agents import initialize_agent, Tool, AgentType
from langchain.chat_models import ChatOpenAI

def calculator(expression: str) -> str:
    return str(eval(expression))

tools = [
    Tool(
        name="Calculator",
        func=calculator,
        description="Use this to do math calculations."
    )
]

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent = initialize_agent(tools, llm, agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION)

response = agent.run("What is 245 multiplied by 12, plus 30?")
print(response)
```

3. Memory Systems

Agents remember previous interactions.

Short-Term Memory

Current conversation.

Long-Term Memory

Stored information across sessions.

Vector Memory

Stores previous conversations as embeddings.

Example of adding short-term conversation memory with LangChain, so the agent remembers earlier turns:

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

memory = ConversationBufferMemory()
conversation = ConversationChain(llm=llm, memory=memory)

conversation.predict(input="My name is Aditya.")
conversation.predict(input="What is my name?")
# -> "Your name is Aditya." (remembered from earlier turn)
```

Benefits:

- Personalized responses
- Better context
- Multi-step reasoning

4. Planning & Reasoning

Instead of directly answering, the agent creates a plan.

Goal: Research AI in Healthcare

```
Search papers -> Summarize  -> Compare  -> Generate Report
```

5. Multi-Agent Systems

Multiple agents collaborate together.

```
Coordinator Agent
  |-- Research Agent
  |-- Writing Agent
  |-- Citation Agent
  +-- Review Agent
```

Advantages:

- Parallel execution
- Modular design
- Better scalability
- Improved accuracy

A simplified LangGraph-style multi-agent setup where a coordinator routes work to a research agent and a writing agent:

```
from langgraph.graph import StateGraph, END

def research_agent(state):
    state["research"] = search_papers(state["topic"])
    return state

def writing_agent(state):
    state["report"] = summarize(state["research"])
    return state

graph = StateGraph(dict)
graph.add_node("research", research_agent)
graph.add_node("write", writing_agent)
graph.set_entry_point("research")
graph.add_edge("research", "write")
graph.add_edge("write", END)

app = graph.compile()
result = app.invoke({"topic": "AI in Healthcare"})
print(result["report"])
```

Frameworks

LangChain

Popular framework for building LLM applications.

Features:

- Prompt Templates
- Chains
- Memory
- Tool Calling
- Agents
- RAG
- Document Loaders
- Output Parsers

Use Cases:

- Chatbots
- RAG
- AI Assistants
- Automation

LangGraph

Designed for complex AI workflows.

Features:

- Stateful Agents
- Graph-based execution
- Human-in-the-loop
- Multi-agent systems
- Cyclic workflows

Ideal for:

- Research Assistants
- AI Workflows
- Enterprise Agents

LlamaIndex

Framework specialized for document understanding.

Features:

- Document Parsing

- Data Connectors
- Indexing
- Retrieval
- RAG Pipelines

Supported Data:

- PDF
- DOCX
- CSV
- SQL
- APIs
- Websites

AI Application Development

APIs

- REST APIs
- HTTP Methods (GET, POST, PUT, DELETE)
- Request & Response
- JSON
- Status Codes
- Authentication Basics

FastAPI Basics

FastAPI is a modern Python web framework used to build the backend of AI applications. It is fast, type-safe, and automatically generates interactive API documentation. Core concepts:

- Project Structure
- Path Parameters
- Query Parameters
- Request Body
- Response Models
- Dependency Injection
- File Upload
- Error Handling
- API Documentation (Swagger/OpenAPI)

A minimal FastAPI app showing path parameters, query parameters, and a request body:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
# Path parameter
@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}

# Query parameter
@app.get("/search")
def search(q: str, limit: int = 5):
    return {"query": q, "limit": limit}

# Request body using a Pydantic model
class Question(BaseModel):
    text: str
    top_k: int = 3

@app.post("/ask")
def ask(question: Question):
    return {"answer": f"You asked: {question.text}"}
```

File upload and error handling are essential for a document-based AI app:

```
from fastapi import FastAPI, UploadFile, File, HTTPException
app = FastAPI()

@app.post("/upload")
async def upload_pdf(file: UploadFile = File(...)):
    if not file.filename.endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Only PDF files are allowed")

    contents = await file.read()
    with open(f"uploads/{file.filename}", "wb") as f:
        f.write(contents)

    return {"filename": file.filename, "status": "uploaded"}
```

Run the app with Uvicorn, and Swagger docs appear automatically at /docs:

```
uvicorn main:app --reload
# Swagger UI:  http://127.0.0.1:8000/docs
```

Vector Databases (Practical Implementation)

Using FAISS or ChromaDB

Operations:

- Store embeddings, Retrieve similar documents
- Update index, Delete documents

Wiring a vector database into a FastAPI endpoint so users can search documents over HTTP:

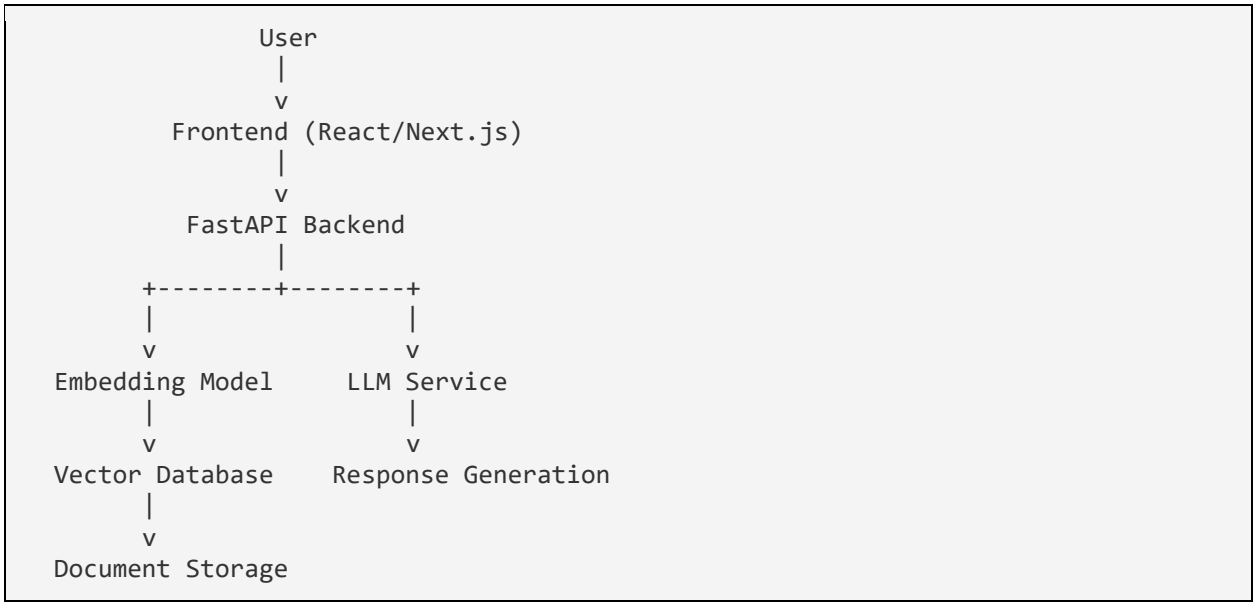
```
from fastapi import FastAPI
from pydantic import BaseModel
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

app = FastAPI()
model = SentenceTransformer("all-MiniLM-L6-v2")
index = faiss.IndexFlatL2(384)
stored_chunks = []

class SearchRequest(BaseModel):
    query: str
    top_k: int = 3

@app.post("/search")
def search(req: SearchRequest):
    query_vector = model.encode([req.query])
    distances, indices = index.search(np.array(query_vector), req.top_k)
    results = [stored_chunks[i] for i in indices[0] if i < len(stored_chunks)]
    return {"results": results}
```

AI Application Architecture



Project Assignment 1: AI Document Search API

Objective: Build a FastAPI application that allows users to upload PDF documents and perform semantic search using embeddings and a vector database.

Features

- Upload PDF documents, Extract text from PDFs
- Generate embeddings, Store embeddings in FAISS or ChromaDB
- Search documents using natural language queries
- Return the most relevant document chunks through a FastAPI API

Tech Stack

- Backend: FastAPI
- Embedding Model: all-MiniLM-L6-v2
- Vector Database: FAISS or ChromaDB
- Document Parser: PyPDF

Deliverables

- Working FastAPI application, Semantic search functionality

Project Assignment 2: AI Research Assistant API

Objective: Build a simple AI assistant using FastAPI that can answer questions, summarize text, and maintain conversation history.

Features

- FastAPI REST API
- Ask questions from uploaded text or documents
- Generate text summaries
- Maintain chat history (memory)
- Use LangChain or LangGraph for tool calling
- Return AI-generated responses with source references

Tech Stack

- Backend: FastAPI
- Framework: LangChain or LangGraph
- LLM: Gemma / Gemini API
- Embedding Model: all-MiniLM-L6-v2 (optional)

Deliverables

- Working AI Assistant API
- API documentation (Swagger/OpenAPI)